

Java Interview Questions & Answers

Core Java

1. Difference between HashMap, Hashtable, and ConcurrentHashMap

HashMap

- Not Thread Safe
- Allows one null key and multiple null values
- Faster

Hashtable

- Thread Safe (synchronized)
- Does not allow null key or value
- Slower

ConcurrentHashMap

- Thread Safe
- Better performance than Hashtable
- Allows concurrent read/write operations

Example

Use HashMap in single-threaded applications.

Use ConcurrentHashMap in multi-threaded applications.

2. How equals() and hashCode() work together?

equals()

Checks object equality.

hashCode()

Returns hash value for object.

Rule

If two objects are equal using equals(), they must have same hashCode().

Example

```
Student s1 = new Student(1);
Student s2 = new Student(1);

s1.equals(s2); // true
s1.hashCode() == s2.hashCode(); // true
```

Used heavily in HashMap and HashSet.

3. Difference between Shallow Copy and Deep Copy

Shallow Copy

Copies object reference.

Changes in child object affect original object.

Deep Copy

Creates completely new object.

Changes do not affect original object.

Example

```
Employee e2 = e1; // Shallow

Employee e2 = new Employee(e1); // Deep
```

4. How to Prevent Deadlock?

Deadlock

Occurs when two threads wait forever for each other.

Prevention

- Lock resources in same order
- Use timeout locks
- Avoid nested locks
- Use ReentrantLock

Example

Always acquire Lock A before Lock B.

5. volatile vs synchronized

volatile

- Guarantees visibility
- Does not provide thread safety

synchronized

- Provides visibility
- Provides thread safety

Example

```
volatile boolean flag = true;
```

Use synchronized when multiple threads modify shared data.

Spring Boot

6. Difference between @Component, @Service, @Repository, @Controller

@Component

Generic Spring Bean.

@Service

Business Logic Layer.

@Repository

Database Layer.

@Controller

Handles Web Requests.

Example

```
@Service
public class UserService {}
```

7. What is Spring Boot Auto Configuration?

Spring Boot automatically configures beans based on dependencies present in classpath.

Example

If spring-boot-starter-data-jpa exists,
Spring automatically creates:

- DataSource
- EntityManager
- TransactionManager

No manual configuration required.

8. Global Exception Handling

Use @ControllerAdvice.

Example

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handle(Exception ex){
        return ResponseEntity.badRequest()
            .body(ex.getMessage());
    }
}
```

Benefit

Centralized exception handling.

9. @RequestParam vs @PathVariable vs @RequestBody

@RequestParam

```
GET /users?id=10
```

```
@RequestParam Long id
```

@PathVariable

```
GET /users/10
```

```
@PathVariable Long id
```

@RequestBody

```
POST /users
```

```
@RequestBody User user
```

Used to receive JSON data.

10. Spring Profiles

Used to manage multiple environments.

Example

application-dev.properties

application-prod.properties

```
spring.profiles.active=dev
```

Hibernate / JPA

11. Hibernate Entity States

Transient

Object created but not saved.

```
User u = new User();
```

Persistent

Managed by Hibernate.

```
entityManager.persist(u);
```

Detached

Removed from persistence context.

Removed

Scheduled for deletion.

12. Lazy Loading

Related data loads only when needed.

Example

```
@OneToMany(fetch = FetchType.LAZY)
```

Problem

LazyInitializationException

Solution

- Use Fetch Join
 - Use Transactional
 - DTO Projection
-

13. Criteria API vs JPQL

JPQL

```
SELECT u FROM User u
```

Easy to read.

Criteria API

Programmatically build queries.

```
CriteriaBuilder cb;
```

Useful for dynamic queries.

14. CascadeType.ALL vs orphanRemoval

CascadeType.ALL

Operations propagate to child.

```
cascade = CascadeType.ALL
```

orphanRemoval

Deletes child if removed from collection.

```
orphanRemoval = true
```

15. Pagination and Sorting

Example

```
Pageable pageable =  
    PageRequest.of(0,10,  
        Sort.by("name"));  
  
Page<User> users =  
    userRepo.findAll(pageable);
```

Used for large datasets.

Microservices

16. Monolithic vs Microservices

Monolithic

Single application.

Advantages:

- Easy development
- Easy deployment

Disadvantages:

- Hard to scale
-

Microservices

Multiple independent services.

Advantages:

- Independent deployment
- Better scalability

Disadvantages:

- Complex management
-

17. Inter-Service Communication

REST API

Most common.

Kafka

Asynchronous communication.

gRPC

High performance communication.

Example:

Order Service → Kafka → Notification Service

18. API Gateway

Single entry point for all requests.

Responsibilities

- Routing
- Authentication
- Rate Limiting
- Logging

Example:

Spring Cloud Gateway

19. Configuration Management

Use Spring Cloud Config Server.

Benefits:

- Centralized configuration
 - Environment-specific configs
-

20. Circuit Breaker

Prevents cascading failures.

Example

Order Service calls Payment Service.

If Payment Service is down:

Circuit Breaker opens and returns fallback response.

Tool

Resilience4j

Cloud & DevOps

21. Docker Image vs Docker Container

Docker Image

Blueprint of application.

Docker Container

Running instance of image.

Example

Image → Java App

Container → Running Java App

22. Kubernetes Scaling

Uses Horizontal Pod Autoscaler (HPA).

Automatically increases pods based on CPU or memory usage.

23. Service Discovery

Allows services to find each other.

Tools:

- Eureka
 - Kubernetes Service Discovery
-

24. Blue-Green vs Rolling Deployment

Blue-Green

Two environments.

Blue = Current

Green = New Version

Switch traffic after testing.

Rolling Deployment

Updates servers one by one.

No downtime.

25. Monitoring Spring Boot Applications

Tools:

- Spring Boot Actuator
- Prometheus
- Grafana
- ELK Stack

Common Metrics:

- CPU
 - Memory
 - Response Time
 - Error Rate
-

26. CI/CD

CI

Continuous Integration

Automatically build and test code.

CD

Continuous Deployment

Automatically deploy application.

Example

Developer Pushes Code

↓

Jenkins Build

↓

Run Tests

↓

Docker Build

↓
Deploy to Kubernetes

SQL

27. Department Wise Average Salary

```
SELECT department,  
AVG(salary)  
FROM employee  
GROUP BY department;
```

28. INNER JOIN vs LEFT JOIN vs RIGHT JOIN vs FULL JOIN

INNER JOIN

Returns matching records.

LEFT JOIN

Returns all records from left table.

RIGHT JOIN

Returns all records from right table.

FULL JOIN

Returns all records from both tables.

29. SQL Query Optimization

Techniques

- Create Indexes
 - Avoid SELECT *
 - Use LIMIT
 - Optimize JOINS
 - Analyze Execution Plan
-

30. ACID Properties

Atomicity

All or nothing.

Consistency

Data remains valid.

Isolation

Transactions don't interfere.

Durability

Data remains after commit.

31. Clustered vs Non-Clustered Index

Clustered Index

Actual data stored in index order.

Only one per table.

Non-Clustered Index

Separate structure pointing to data.

Multiple allowed.

DSA

32. Reverse Linked List

Iterative

Use:

- prev
- current
- next

Time Complexity: $O(n)$

33. First Non-Repeating Character

Use LinkedHashMap.

Count frequency.

Return first character with count 1.

Example:

Input:

aabbcd

Output:

c

34. Longest Common Prefix

Input:

flower

flow

flight

Output:

fl

Approach:

Compare characters one by one.

35. LRU Cache

Uses:

- HashMap
- Doubly Linked List

Operations:

get() → O(1)

put() → O(1)

36. REST API for Last N Orders

```
@GetMapping("/orders")
public List<Order> getOrders(
    @RequestParam int limit){

    return orderRepo
        .findTopNOrders(limit);
}
```

Use Pageable in JPA for better implementation.